

program exits, so you should avoid putting any program-ending critical code in this method.

Garbage Collection

When an object is deleted, either through the reference count going to zero or through the explicit use of a `del` statement, as the programmer you should consider the object inaccessible.

However, the specific implementation of the garbage collector is completely up to Python. The details of the algorithm that decides when the actual garbage collection code runs are not important to you as the programmer. It may run when your program instantiates an object and Python needs to allocate memory, or at random times, or at certain scheduled times. The algorithm may change from one release of Python to another. Whichever it is, Python will take care of garbage collection, and you needn't worry about the specifics.

Class Variables

I have talked extensively about how instance variables are defined in a class and how each object instantiated from a class gets its own set of all the instance variables. The prefix `self.` is used to identify each instance variable. However, you can also create *class variables* at the class level.

class variable

A variable that is defined in and owned by a class. Only one of each class variable exists, independent of how many instances of that class are created.

You create a class variable with an assignment statement, which by convention is placed between the `class` statement and the first `def` statement, like so:

```
class MyDemoClass():  
    myClassVariable = 0 # create a class variable and assign 0 to it
```